SOFTWARE ENGINEERING LAB 7

FUZZING FOR INPUT VALIDATION BUGS

Name: Varshitha S
SRN: PES1UG24AM454
Class: AIML – Section H

1. Objective

To identify input-validation bugs in a Python application using property-based fuzz testing with the Hypothesis framework, and to fix the detected issues by implementing appropriate input validation.

2. Introduction

Fuzz testing is a software testing technique that provides a program with a wide range of inputs to identify unexpected behavior, crashes, and input-validation errors.

In this lab, the Hypothesis framework was used to generate different inputs and test three functions in the Movie Ticket application: `parse_seat()`, `total_price()`, and `average_rating()`.

3. Tools and Technologies Used

- Python 3.13.7
- Pytest 9.2.0
- Hypothesis 6.170.15
- Windows PowerShell

4. Application Description

The Movie Ticket application contains three main functions:

- `parse_seat()`: Extracts the row letter and seat number from a seat identifier.
- `total_price()`: Calculates the total ticket price after adding 18% GST.

- `average_rating()`: Calculates the average of the given ratings.

5. Initial Application Execution

The original application was executed using valid sample inputs.

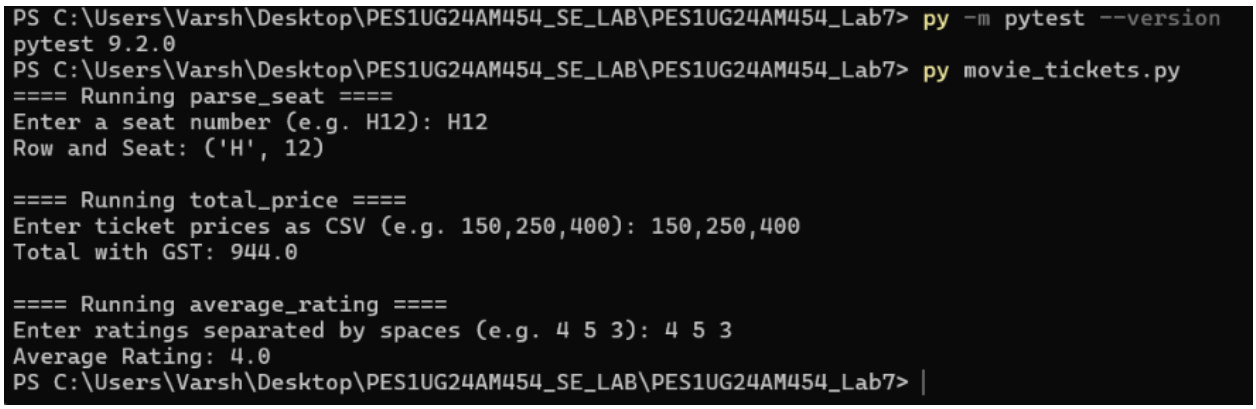
Inputs:

- Seat number: H12
- Ticket prices: 150,250,400
- Ratings: 4 5 3

Output:

- Row and Seat: ('H', 12)
- Total with GST: 944.0
- Average Rating: 4.0

Screenshot 1: Original Application Execution



6. Fuzz Testing and Bugs Identified

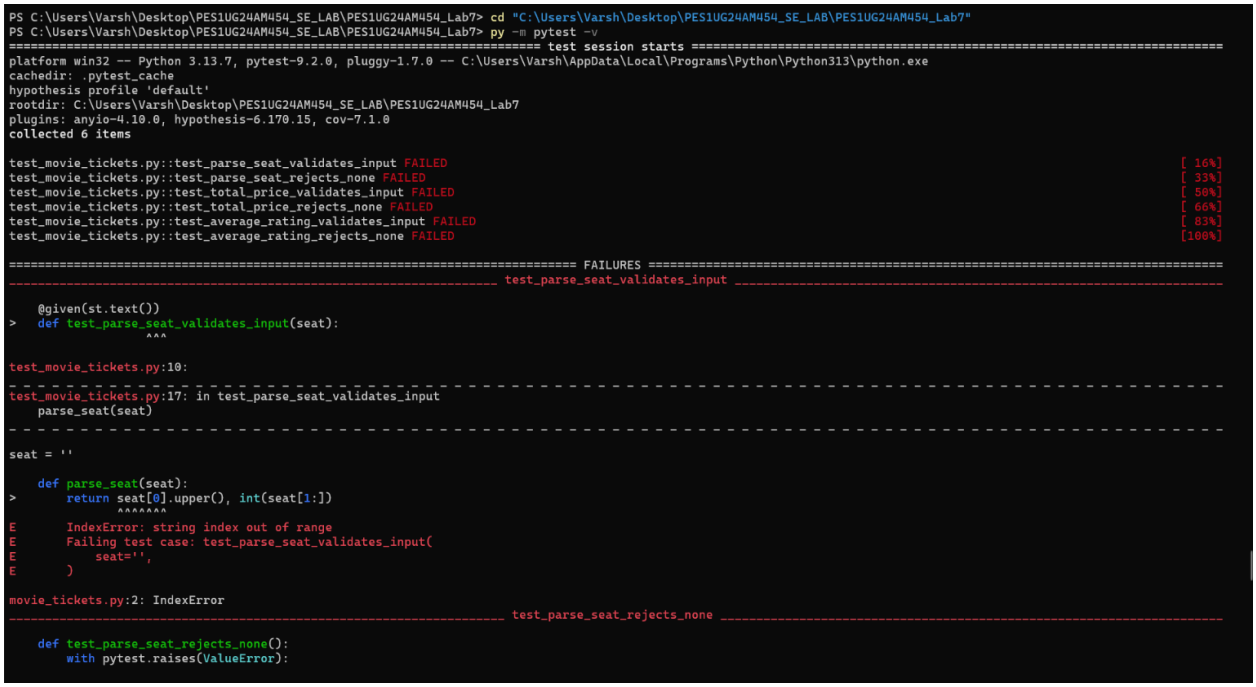
Hypothesis was used with Pytest to generate different inputs and identify validation issues in the application.

The initial fuzz testing identified the following problems:

1. `parse_seat()` produced an IndexError for empty input and a TypeError for None.
2. `total_price()` accepted negative ticket prices and produced an AttributeError for None.
3. `average_rating()` accepted ratings outside the valid range of 1 to 5 and produced an AttributeError for None.

The initial test execution resulted in **6 failed tests**, revealing the input-validation issues.

Screenshot 2: Initial Fuzz Testing Failures



```
PS C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7> cd "C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7"
PS C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7> py -m pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Varsh\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7
plugins: anyio-4.10.0, hypothesis-6.170.15, cov-7.1.0
collected 6 items

test_movie_tickets.py::test_parse_seat_validates_input FAILED            [ 16%]
test_movie_tickets.py::test_parse_seat_rejects_none FAILED               [ 33%]
test_movie_tickets.py::test_total_price_validates_input FAILED           [ 50%]
test_movie_tickets.py::test_total_price_rejects_none FAILED              [ 66%]
test_movie_tickets.py::test_average_rating_validates_input FAILED        [ 83%]
test_movie_tickets.py::test_average_rating_rejects_none FAILED           [100%]

================================== FAILURES ===================================
_____________________ test_parse_seat_validates_input _____________________

    @given(st.text())
>   def test_parse_seat_validates_input(seat):
                   ^^^

test_movie_tickets.py:10:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
test_movie_tickets.py:17: in test_parse_seat_validates_input
    parse_seat(seat)
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

seat = ''

    def parse_seat(seat):
>       return seat[0].upper(), int(seat[1:])
               ^^^^^^^
E       IndexError: string index out of range
E       Failing test case: test_parse_seat_validates_input(
E           seat='',
E       )

movie_tickets.py:2: IndexError
_____________________ test_parse_seat_rejects_none _____________________

    def test_parse_seat_rejects_none():
        with pytest.raises(ValueError):
```



```
test_movie_tickets.py:22:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

seat = None

    def parse_seat(seat):
>       return seat[0].upper(), int(seat[1:])
               ^^^^^^^
E       TypeError: 'NoneType' object is not subscriptable

movie_tickets.py:2: TypeError
_____________________ test_total_price_validates_input _____________________

    @given(st.lists(st.integers(min_value=-1000, max_value=1000), min_size=1))
>   def test_total_price_validates_input(prices):
                   ^^^

test_movie_tickets.py:27:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

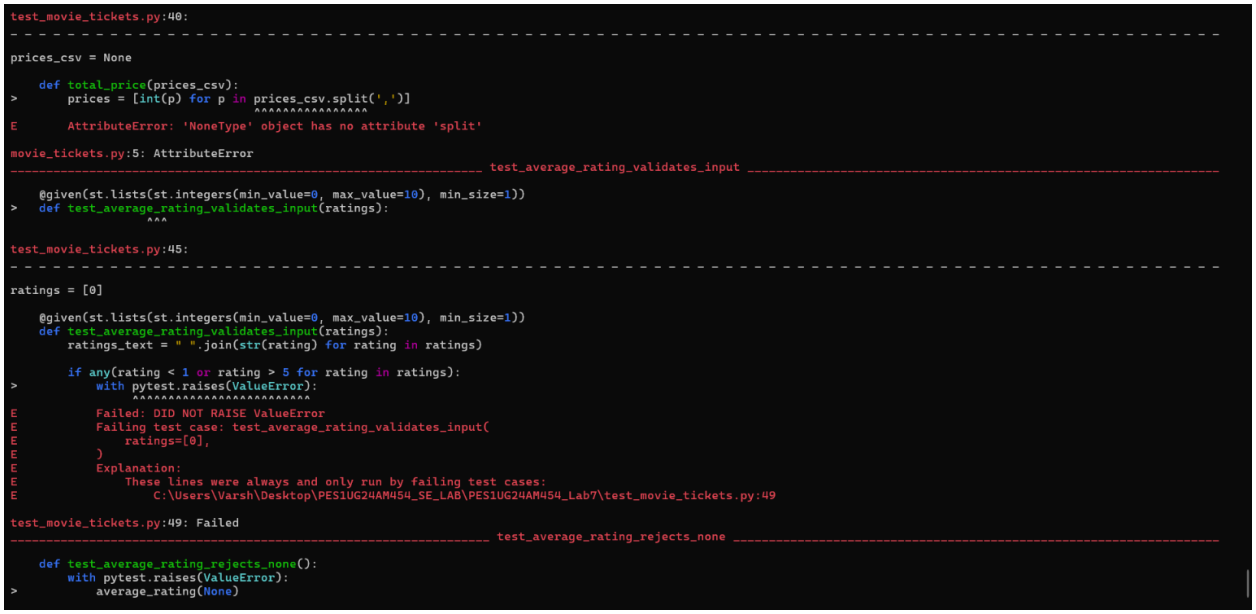
prices = [-1]

    @given(st.lists(st.integers(min_value=-1000, max_value=1000), min_size=1))
    def test_total_price_validates_input(prices):
        prices_csv = ",".join(str(price) for price in prices)

        if any(price < 0 for price in prices):
>           with pytest.raises(ValueError):
                ^^^^^^^^^^^^^^^^^^^^^^^^^^
E           Failed: DID NOT RAISE ValueError
E           Failing test case: test_total_price_validates_input(
E               prices=[-1],
E           )
E           Explanation:
E               These lines were always and only run by failing test cases:
E                   C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7\test_movie_tickets.py:31

test_movie_tickets.py:31: Failed
_____________________ test_total_price_rejects_none _____________________

    def test_total_price_rejects_none():
        with pytest.raises(ValueError):
>           total_price(None)
```





7. Bug Fixes and Implementation

The application was modified to handle invalid inputs using appropriate validation checks.

The following improvements were implemented:

- Seat identifiers are validated to ensure they contain one alphabetic character followed by a positive number.
- Ticket prices are validated to ensure they are non-negative integers.
- Ratings are validated to ensure they are integers between 1 and 5.
- Invalid and missing inputs are handled by raising ValueError with appropriate error messages.

These changes prevent unexpected exceptions and ensure that the functions process only valid inputs.

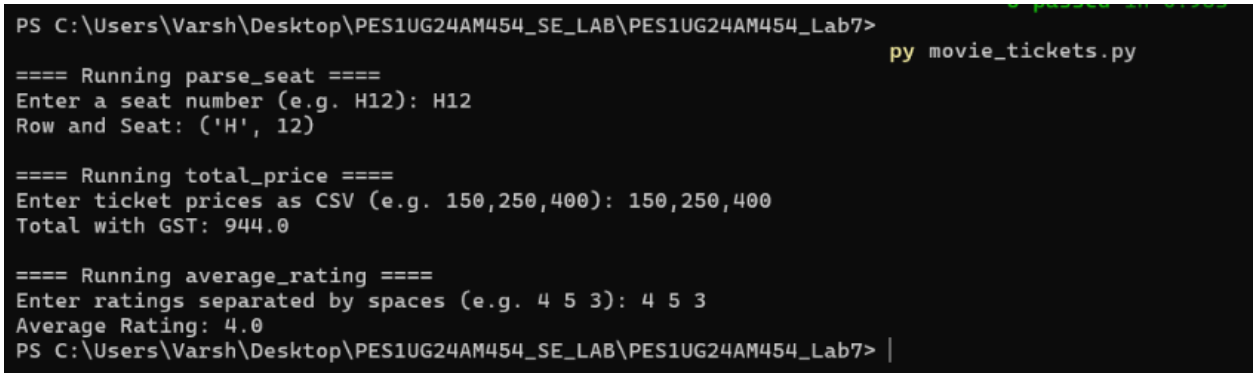
8. Final Application Execution

After implementing the fixes, the application was executed again using valid sample inputs.

Output:

- Row and Seat: ('H', 12)
- Total with GST: 944.0
- Average Rating: 4.0

Screenshot 3: Fixed Application Execution



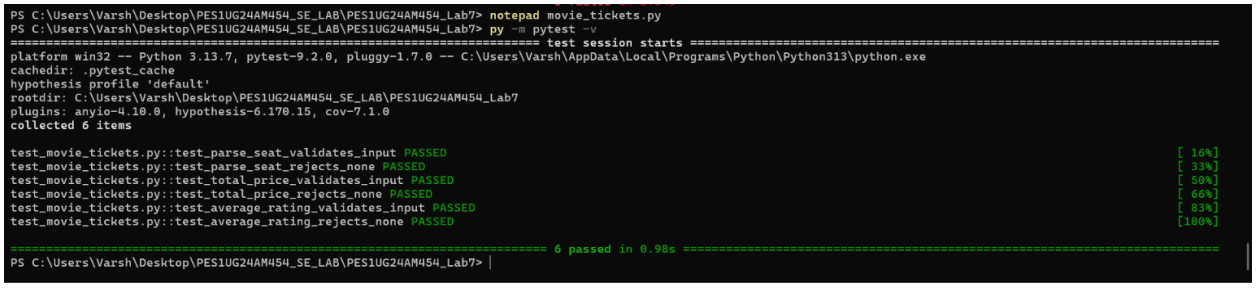
9. Final Fuzz Testing Results

The Hypothesis test suite was executed again after fixing the application.

Final Result: 6 passed in 0.98 seconds.

All six test cases passed successfully, confirming that the identified input-validation issues were addressed.

Screenshot 4: Final Fuzz Testing Results



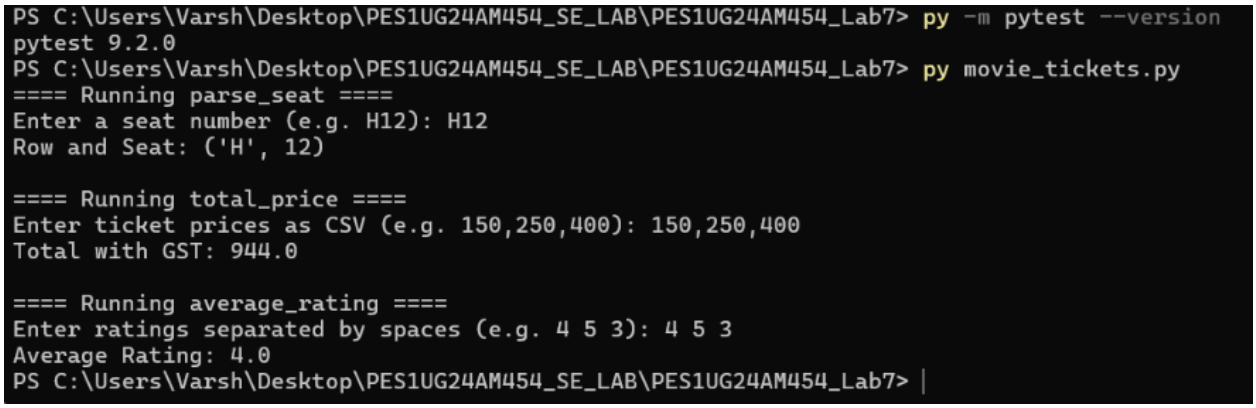
10. Conclusion

The lab demonstrated the use of property-based fuzz testing with Hypothesis to identify input-validation bugs in a Python application. The testing process revealed issues involving empty inputs, None values, negative prices, and invalid ratings.

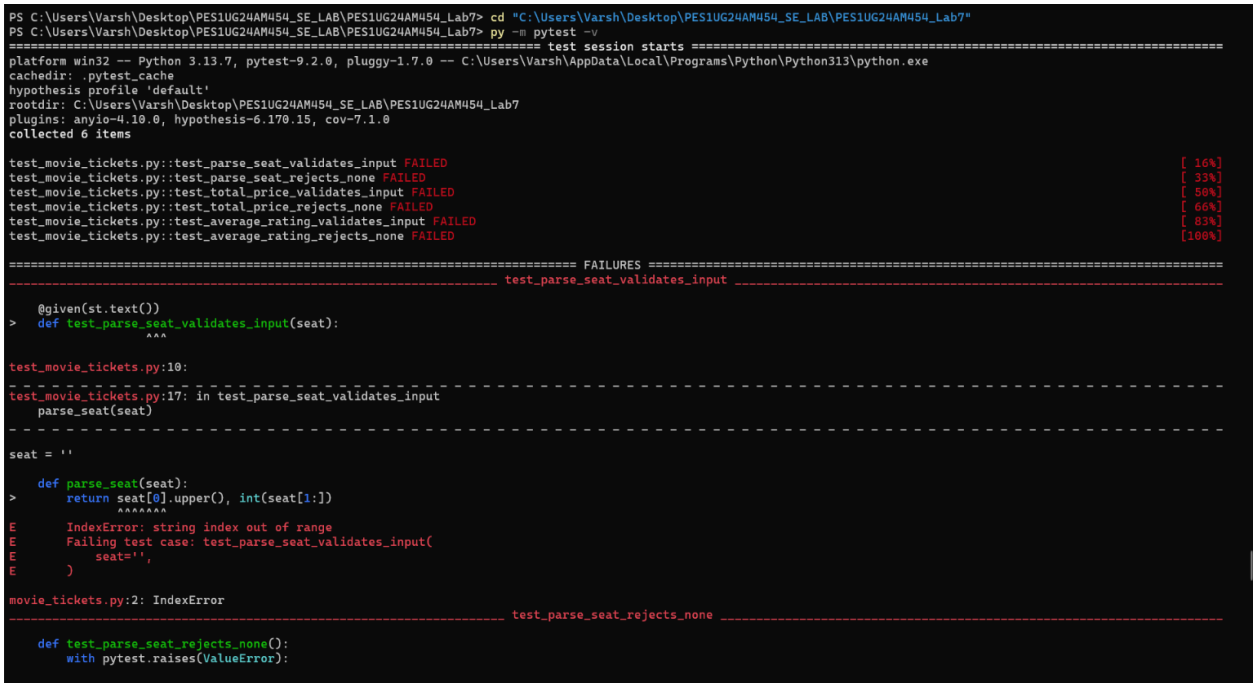
After implementing appropriate validation checks, all six test cases passed successfully. This exercise demonstrated how fuzz testing can improve application reliability and robustness by identifying unexpected input-related errors.

SCREENSHOTS:

01_Original_Application.png



02_Fuzz_Test_Failures.png



```
PS C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7> cd "C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7"
PS C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7> py -m pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Varsh\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Varsh\Desktop\PES1UG24AM454_SE_LAB\PES1UG24AM454_Lab7
plugins: anyio-4.10.0, hypothesis-6.170.15, cov-7.1.0
collected 6 items

test_movie_tickets.py::test_parse_seat_validates_input FAILED            [ 16%]
test_movie_tickets.py::test_parse_seat_rejects_none FAILED               [ 33%]
test_movie_tickets.py::test_total_price_validates_input FAILED           [ 50%]
test_movie_tickets.py::test_total_price_rejects_none FAILED              [ 66%]
test_movie_tickets.py::test_average_rating_validates_input FAILED        [ 83%]
test_movie_tickets.py::test_average_rating_rejects_none FAILED           [100%]

================================== FAILURES ===================================
_____________________ test_parse_seat_validates_input _____________________

    @given(st.text())
>   def test_parse_seat_validates_input(seat):
                   ^^^

test_movie_tickets.py:10:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
test_movie_tickets.py:17: in test_parse_seat_validates_input
    parse_seat(seat)
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

seat = ''

    def parse_seat(seat):
>       return seat[0].upper(), int(seat[1:])
               ^^^^^^^

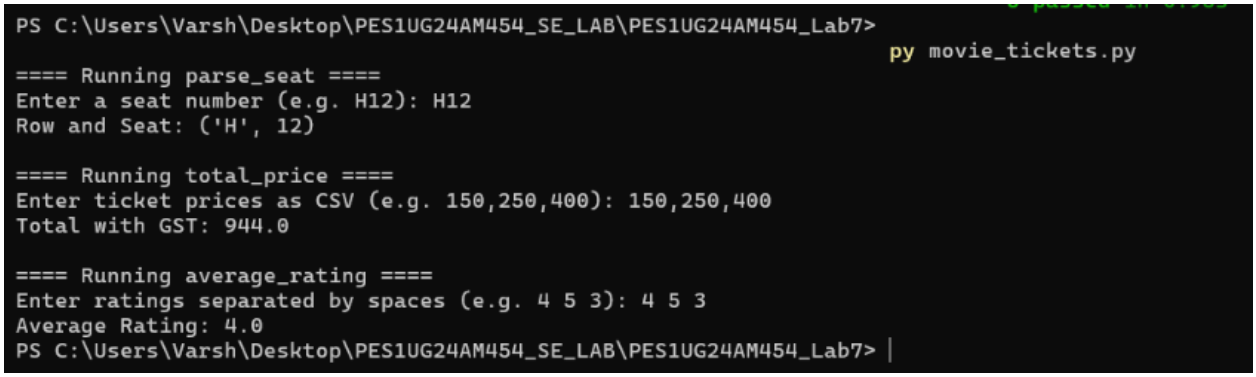
E       IndexError: string index out of range
E       Failing test case: test_parse_seat_validates_input(
E           seat='',
E       )

movie_tickets.py:2: IndexError
_____________________ test_parse_seat_rejects_none _____________________

    def test_parse_seat_rejects_none():
        with pytest.raises(ValueError):
```

```
test_movie_tickets.py:22:
- - - -
```

03_Fixed_Application.png



04_Final_Fuzz_Test_Results.png

